Linked Lists

We are continuing to learn of implementations of the List ADT. This time, we are learning of an implementation using pointer-based data structures instead of arrays. That's right, we can create our own structures to hold groups of data, instead of just using arrays. First let's get familiar with this concept of cobbling together pointer-based structures to form a data structure.

The pointer-based structure is often what we call the **node**. The node stores our data and often links to *other* nodes (using pointers) that are part of the collection. In this chapter, we'll learn about utilizing nodes to create a sequence of elements to implement the List ADT. We'll first tackle the singly-linked list, which can implement the Stack and FIFO Queue operations in *constant* time.

SLList: Singly-linked List

A `SLList` is a sequence of nodes chained together by pointers. Each node `u` stores a data value `u.x` and a reference to the next node in line, `u.next`. The last node in the sequence doesn't have a next, so for a last node `w`, `w.next = null`. A visual representation of the `SLList` is shown in fig. 15.

```
template<class T>
class Node {
    T x;
    Node *next;

    Node(T x0) {
        x = x0;
        next = NULL;
    }
    ...
}
```

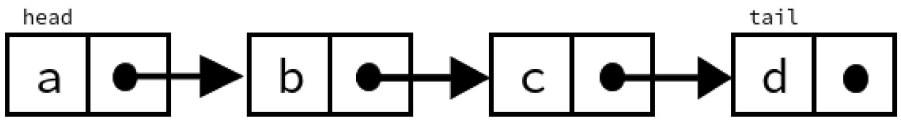


Figure 15: Visual representation of a `SLList`

For efficiency, an `SLList` uses 2 variables `head` and `tail` to keep track of the first and last node in the sequence. We also keep track of

the number of elements in the list, `n`.

```
template<class T>
class SLList {
    Node<T> *head;
    Node<T> *tail;
    int n;
}
```

`push(x)` and `pop()`

An `SLList` can efficiently implement the Stack operations `push(x)` and `pop()` by adding and removing elements at the `head`. The `push(x)` operation creates a new `Node` `u`, sets `u.x` to `x`, sets `u.next` to the current `head`, updates `head` to the new node `u`, and increments `n`. In the case where the list is empty, we also set the `tail` to the new node. We illustrate the `push(x)` operation in fig. 16

```
T push(T x) {
    Node *u = new Node(x);
    u->next = head;
    head = u;
    if (n == 0) {
        tail = u;
    }
    n++;
    return x;
}
```

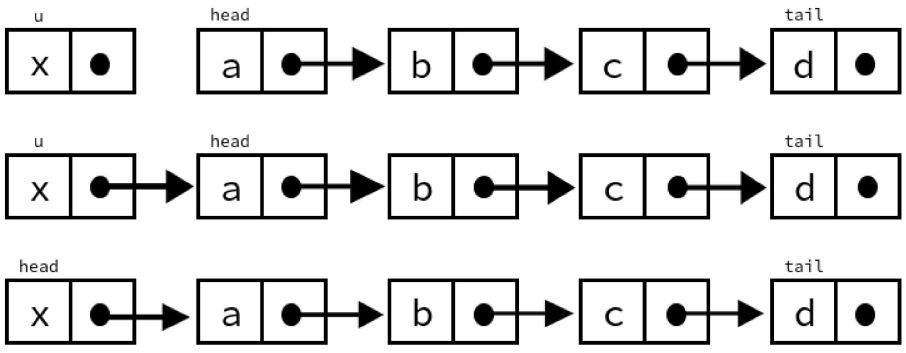


Figure 16: A `push(x)` operation on a `SLList`

The `pop()` operation is similar, we store the current value of `head.x` in `y`, update `head` to `head.next`, decrement `n`, and return the

stored value `y`. Again, when the list becomes empty, we set the `tail` to `NULL`.

> Don't forget to free up memory with the `delete` keyword!

```
T pop() {
    if (n == 0)
        return NULL;
    T y = head->x;
    Node *u = head;
    head = head->next;
    delete u;
    n--;
    if (n == 0)
        tail = NULL;
    return y;
}
```

Simple, no loops, we can see that it runs in constant time—$O(1)$.

The `SLList` can *also* implement the FIFO Queue operations `add(x)` and `remove()` in constant time.

`add(x)` and `remove()`

In a FIFO Queue, `add(x)` operations are made at the `tail` of our list. This is easily implemented by creating a node `u` with value `x`, setting `tail.next` to `u`, updating `tail` point to `u` and finally incrementing `n`. Do note that when adding to an empty queue, we have to set both `tail` and `head` to the same node.

The `remove()` operation is made at the `head` of the list, and is completely identical to the `pop()` operation.

Both operations run in constant time as well.

If you notice, we have almost implemented the Deque ADT as well. The only missing operation is `removeLast()`. To implement that, we'll have to reach a node `w` such that `w.next` = `tail` and update that to point to `NULL`. In other words, we have to reach the second to the last node in the sequence to update it to point to `NULL`. Unfortunately, if you *also* notice, the only way to reach the *second* to the last node is to start from the `head` and go through almost the entire list. This would take `n` - 2 steps. This is a limitation of the singly-linked list. What can

we do? Wouldn't it be *so* convenient if we could start from the tail and work back?

DLList: Doubly-linked List

The doubly-linked list does just that. It is an *improvement* over the `SLList` in that our node now has *2* pointers— `next` and `prev`. The `next` pointer points to the next item in the list, and the `prev` pointer points to the previous.

```
template<class T>
class Node {
    T x;
    Node *next;
    Node *prev;
};
```

When we were implementing the `SLList` we had to deal with setting pointers to `NULL` and being very careful about them. In a `DLList`, the number of these special cases of setting pointers to `NULL` increase by a lot. To help with dealing with these special cases, we introduce a `dummy` node. This is a node that does not carry any data, but rather acts as a placeholder—so that we don't have to deal with `NULL`s. With this `dummy` node, our list is linked into a cycle (fig. 17).

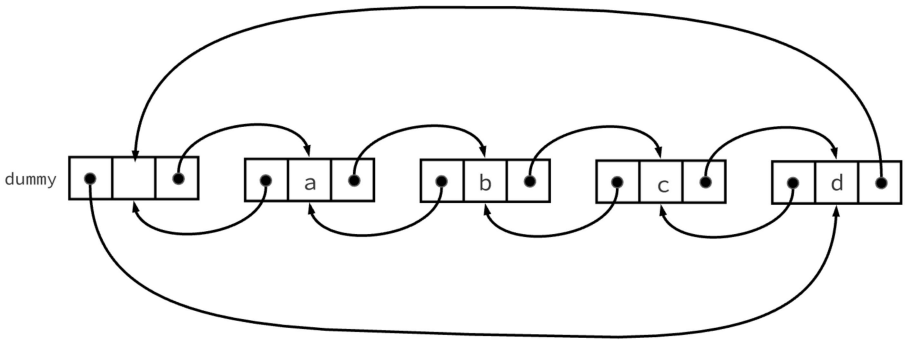


Figure 17: A `DLList` containing a, b, c, d

```
template<class T>
class DLList {
    Node dummy;
    int n;

    DLList() {
        dummy.next = &dummy;
        dummy.prev = &dummy;
        n = 0;
    }
}
```

`get(i)` and `set(i, x)`

Finding a node with a particular index is easy. We can start at the `dummy` node and go through the `head` (that is, `dummy.next`) or through the `tail` (that is, `dummy.prev`). This is a technique quite similar to the `ArrayDeque`, we take the shorter path to the node we want. If the index we want is closer to the `head`, we go through the `head`. Otherwise, we go through the `tail` and work backwards.

```
Node* getNode(int i) {
    Node *p;
    if (i < n / 2) {
        p = dummy.next;
        for (int j = 0; j < i; j++) {
            p = p->next;
        }
    } else {
        p = dummy.prev;
        for (int j = n; j > i; j--) {
            p = p->prev;
        }
    }
    return p;
}
```

With the `getNode(i)` method, the `get(i)` and `set(i, x)` become easy. We just have to find the `i` th node and get or set its value.

```
T get(int i) {
    return getNode(i)->x;
}

T set(int i, T x) {
    Node *u = getNode(i);
    T y = u->x;
    u->x = x;
    return y;
}
```

`add(i, x)`

Before we implement the `add(i, x)`, let us consider that we have a node `w` in a `DLList` and we want to insert a new node `u` before `w`. This is easily done by setting `u.next` to `w`, `u.prev` = `w.prev`, and fixing the other links by pointing `u.prev.next` and `u.next.prev` to `u`. This is illustrated in fig. 18. Thanks to our `dummy` node, we don't have to worry about `w.prev` or `w.next` being `NULL`. We'll name the method described here `addBefore(w, x)`, which inserts a node with value `x` before the node `w`.

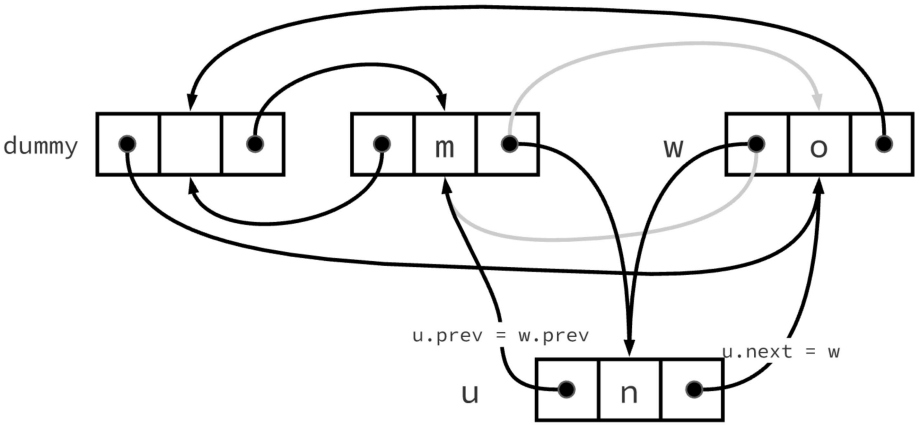


Figure 18: Adding a node `u` before node `w` in a `DLList`

With `addBefore(w, x)`, the `add(i, x)` operation is now trivial.

```
void add(int i, T x) {
    addBefore(getNode(i), x);
}
```

Isn't that nice? We're splitting code into functions that do one thing, and they do it well. We should strive to compartmentalize our code into functions like these. It helps out a ton.

`remove(i)`

Removing a node is *easier* than adding one, all we have to do is update the links to *skip* the removed node. That's it. Having a `dummy` node helps here, we don't have to think of edge cases to set links to `NULL`. It just works.

Analysis

All the methods, `add(i, x)`, `remove(i)`, `set(i, x)`, and `get(i)` are limited by locating the relevant node. We have to spend $O(1 + \min(\texttt{i}, \texttt{n - i}))$ time to find the relevant node with the `getNode(i)` operator for all these operations. If we ignore the time `getNode(i)` takes, we actually take constant time. We summarize the operations' time complexity in tbl. 5.

Table 5: Summarized time complexity for a `DLList`

Operation	Time Complexity
`get(i)`	$O(1 + \min(\texttt{i}, \texttt{n - i}))$
`set(i, x)`	$O(1 + \min(\texttt{i}, \texttt{n - i}))$
`add(i, x)`	$O(1 + \min(\texttt{i}, \texttt{n - i}))$
`remove(i)`	$O(1 + \min(\texttt{i}, \texttt{n - i}))$